SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SHARWIN 192521207 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:sharwin

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

   i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved. c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital Doctor–Shift Assignment Using Backtracking Search
Problem Statement
A hospital has three doctors:

- **D1 • D2**
- **D3**

and three shifts:

- **Morning (M)**
- **Afternoon (A)**
- **Night (N)**

Each doctor must be assigned exactly one shift.
The constraints are:

1. D1 cannot work Night.
2. D2 must work before D3.
3. Only one doctor can work each shift.
4. D3 cannot work Morning.
5. Every doctor must receive exactly one shift.

The problem can be solved using **Backtracking Search**, which is a systematic way of trying assignments and going back whenever a constraint is violated.

Step 1: Define Variables The
variables are:
\[D1,\ D2,\ D3 \]
Each variable represents the shift assigned to that doctor.

Step 2: Define Domains
Initially, every doctor can potentially work any shift: \[D1 =
\{M,A,N\} \]\[D2 = \{M,A,N\} \]\[D3 = \{M,A,N\} \] After
applying individual constraints:
For D1
D1 cannot work Night.
\[D1=\{M,A\} \]
For D3
D3 cannot work Morning.
\[D3=\{A,N\} \] D2
initially has:
\[D2=\{M,A,N\} \]

Step 3: Constraint D2 Before D3
The order of shifts is: \[M < A <
N \] Therefore:
\[Shift(D2)<Shift(D3) \] Possible
combinations are:
D2 D3 Valid?
M A Yes
M N Yes
D2 D3 Valid?
A M No
A N Yes
N M No
N A No

Step 4: Backtracking Search We can
assign the doctors one at a time.
We will use the order:
\[D1 \rightarrow D2 \rightarrow D3 \]

Step 1 – Assign D1 Try:
\[D1=M \] Check
constraint:

- D1 cannot work Night → D1 is Morning → **Satisfied**
- No other doctor has been assigned yet.

Therefore: \[
D1=M \] is
valid.
Current assignment:
D1 → Morning D2
→ ?
D3 → ?

Step 2 – Assign D2 Try:
\[D2=M \]
But D1 is already working Morning.
Constraint:
Only one doctor per shift.
Therefore: \[D2=M \] is
invalid.
Backtrack
Try the next value:
\[D2=A \] Check:

- Morning is occupied by D1 → A is free.
- D2 can work Afternoon.
- D2 must work before D3.

Therefore: \[D2=A
\] is currently valid.
Current assignment:
D1 → Morning D2
→ Afternoon D3
→ ?

Step 3 – Assign D3 D3
cannot work Morning.
Try:
\[D3=M \]
This violates:
D3 cannot work Morning.
Therefore: \[D3=M \] is
rejected. Backtrack and
try:
\[D3=A \]
But D2 is already working Afternoon.
Therefore:
Only one doctor per shift.
So:
\[D3=A \] is
rejected.
Backtrack and try:
\[D3=N \]
Check all constraints:

- D3 is not Morning → ✓
- Night is free → ✓
- D2 = Afternoon
- D3 = Night
- Afternoon < Night → ✓ Therefore: \[D3=N \] is valid.

Final Schedule
Doctor Shift
D1 Morning
D2 Afternoon
D3 Night
Therefore:
\[\boxed{D1=M,\quad D2=A,\quad D3=N} \]

Complete Backtracking Trace

Step	Assignment	Constraint Check	Result
1	D1 = M	D1 ≠ N	✓
2	D2 = M	M already occupied by D1	✗

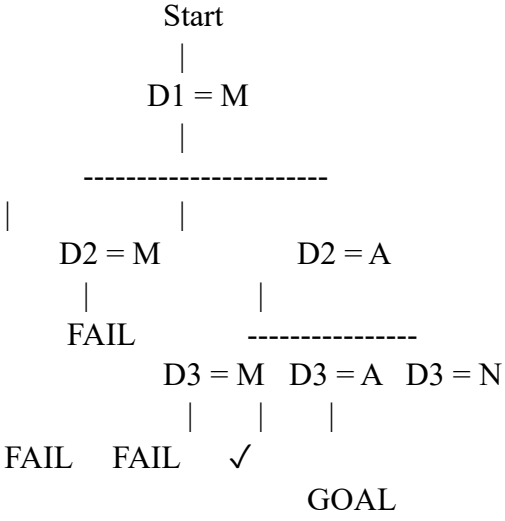
3 D2 = A A is free ✓

4 D3 = M D3 ≠ M ✗

5 D3 = A A occupied by D2 ✗

6 D3 = N D2 < D3 and N is free ✓

7 D1=M, D2=A, D3=N All constraints satisfied ✓ GOAL **Backtracking Tree**



Pseudocode

```
BACKTRACK(assignment):

   IF all doctors are assigned:
      return assignment

   Select an unassigned doctor

   FOR each possible shift:

      IF shift satisfies all constraints:

         Assign shift to doctor

         result = BACKTRACK(assignment)

         IF result is successful:
            return result

         Remove assignment   ← Backtrack

   return failure
```

Python Code

```
doctors = ["D1", "D2", "D3"]shifts = ["Morning", "Afternoon", "Night"]def
is_valid(assignment):    # D1 cannot work Night    if assignment.get("D1") == "Night":
```

return False # D3 cannot work Morning if assignment.get("D3") == "Morning": return False # Only one doctor per shift assigned_shifts = list(assignment.values()) if len(assigned_shifts) != len(set(assigned_shifts)): return False # D2 must work before D3 if "D2" in assignment and "D3" in assignment: order = { "Morning": 1, "Afternoon": 2, "Night": 3 } if order[assignment["D2"]] >= order[assignment["D3"]]: return False return Truedef backtracking(assignment):
Expected Final Output Final Schedule:
D1 -> Morning
D2 -> Afternoon D3
-> Night

---

2. 5×5 Grid Search Using Manhattan Distance
Problem Statement
A robot operates in a **5 × 5 grid**.
It must travel from:

- Start = **S**
- Goal = **G**

The robot has the following constraints:

1. It can move Up, Down, Left and Right.
2. Each movement has cost 1.
3. Obstacles cannot be entered.
4. Manhattan Distance is used as the heuristic.
5. The objective is to find an optimal path.

Important Note
The question says **BFS + Manhattan Distance + priority queue**. Strictly speaking:
BFS
BFS uses:
$$ f(n)=g(n) $$ and normally uses a normal FIFO queue.
A*
A* uses:
$$ \boxed{f(n)=g(n)+h(n)} $$ where:

- $g(n)$ = cost from Start to current node
- $h(n)$ = estimated cost from current node to Goal

Since the question specifically asks for **Manhattan Distance heuristic and priority queue**, the appropriate algorithm is **A***.

Assumed 5×5 Grid
Since the question does not provide the actual obstacle positions or grid figure, we can use the following example grid. C1 C2 C3 C4 C5
R1 S . . # .
R2 # . # . .
R3 . . . # .

R4 . # . . . R5
. . . . G
Where:

- S = Start
- G = Goal
- # = Obstacle
- . = Free cell Start: \[S=(1,1) \] Goal:

\[G=(5,5) \]

Manhattan Distance
The Manhattan distance is:
\[\boxed{h(n)=|x_n-x_G|+|y_n-y_G|} \] For
example, for the starting node:
\[S=(1,1) \]
Goal: \[
G=(5,5) \]
Therefore:
\[h(S)=|1-5|+|1-5| \]\[h(S)=4+4 \]\[\boxed{h(S)=8} \]

Evaluation Function A*
uses:
\[\boxed{f(n)=g(n)+h(n)} \]
where: **g(n)**
Actual cost from Start to current node.
h(n)
Estimated cost from current node to Goal.
f(n)
Total estimated solution cost through the node.

Step-by-Step Search
Step 1 – Start Node
Start: \[(1,1) \]
Cost:
\[g=0 \]
Heuristic: \[
h=8 \]
Therefore:
\[f=0+8=8 \] Priority
Queue:
Node g h f
(1,1) 0 8 8

Step 2 – Expand (1,1) Possible
movements:
Right

\[(1,2) \] This
is free.
\[g=1 \]\[h=|1-5|+|2-5|=4+3=7 \]\[f=1+7=8 \]
Down
\[(2,1) \]
This is an obstacle.
Therefore:
\[(2,1)=\text{Rejected} \] Priority
Queue:
Node g h f
(1,2) 1 7 8

Step 3 – Expand (1,2) Possible
movements:

- Left → (1,1), already visited.
- Right → (1,3)
- Down → (2,2)
- Up → outside grid.

(1,3)
\[g=2 \]\[h=|1-5|+|3-5|=4+2=6 \]\[f=2+6=8 \]
(2,2)
\[g=2 \]\[h=|2-5|+|2-5|=3+3=6 \]\[f=2+6=8 \] Priority
Queue:
Node g h f
(1,3) 2 6 8
(2,2) 2 6 8

Step 4 – Expand (1,3) Possible
movements:

- Left → (1,2), visited.
- Right → (1,4), obstacle.
- Down → (2,3), obstacle.
- Up → outside. Therefore:

No useful successor.
Remove (1,3) from consideration.
Queue:
Node g h f
(2,2) 2 6 8

Step 5 – Expand (2,2) Possible
movements:

- Up → (1,2), visited.
- Left → (2,1), obstacle.
- Right → (2,3), obstacle.
- Down → (3,2), free.

For:
\[(3,2) \]\[g=3 \]\[h=|3-5|+|2-5| \]\[h=2+3=5 \]
Therefore: \[f=3+5=8 \] Queue:
Node g h f
(3,2) 3 5 8

Step 6 – Expand (3,2) Possible successors:

- Up → (2,2), visited.
- Left → (3,1), free.
- Right → (3,3), free.
- Down → (4,2), obstacle.

(3,1)
\[g=4 \]\[h=|3-5|+|1-5|=2+4=6 \]\[f=10 \]
(3,3)
\[g=4 \]\[h=|3-5|+|3-5|=2+2=4 \]\[f=8 \] Queue:
Node g h f
(3,3) 4 4 8
(3,1) 4 6 10
A* chooses:
\[(3,3) \] because it has the lowest \(f\).

Step 7 – Expand (3,3) Possible successors:

- Up → (2,3), obstacle.
- Down → (4,3), free.
- Left → (3,2), visited.
- Right → (3,4), obstacle.

For:
\[(4,3) \]\[g=5 \]\[h=|4-5|+|3-5|=1+2=3 \]\[f=5+3=8 \] Queue:
Node g h f
(4,3) 5 3 8
(3,1) 4 6 10

Step 8 – Expand (4,3) Possible successors:

- Up → (3,3), visited.
- Down → (5,3), free.
- Left → (4,2), obstacle.
- Right → (4,4), free.

(5,3)
\[g=6 \]\[h=|5-5|+|3-5|=0+2=2 \]\[f=8 \]
(4,4)
\[g=6 \]\[h=|4-5|+|4-5|=1+1=2 \]\[f=8 \] Queue:

Node g h f
(5,3) 6 2 8
(4,4) 6 2 8
(3,1) 4 6 10

Step 9 – Expand (5,3) Possible successors:

- Up → (4,3)
- Left → (5,2)
- Right → (5,4)
- Down → outside

For:
\[(5,4) \]\[g=7 \]\[h=|5-5|+|4-5|=1 \]\[f=8 \] Queue contains:
Node g h f
(5,4) 7 1 8
(4,4) 6 2 8
(5,2) 7 3 10
(3,1) 4 6 10

Step 10 – Reach Goal From:
\[(5,4) \]
move Right:
\[(5,5)=G \]
Cost: \[
g(G)=8 \]
Heuristic: \[
h(G)=0 \]
Therefore:
\[f(G)=8+0=8 \]
The goal is reached.

Optimal Path
The path found is:
\[\boxed{ (1,1) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (4,3) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5) } \] **Path Diagram**

```
S → → #
    ↓ #
    ↓ → #
#   ↓
#   ↓ →
    ↓ ↓
    → → G
```

More clearly:

S → . # .#
↓ # . .
. ↓ → # .
. # ↓ → .
. . ↓ → G

Total Cost

There are 8 movements.
Each movement costs 1.
\[\boxed{Total\ Cost=8} \]

A* Pseudocode

```
A_STAR(Start, Goal):

    Create priority queue OPEN

    Put Start into OPEN

    g(Start) = 0

    h(Start) = Manhattan Distance

    f(Start) = g(Start) + h(Start)

    WHILE OPEN is not empty:

        Select node with minimum f value

        IF node == Goal:
            return solution path

        Move node to CLOSED

        FOR each valid neighbor:

            IF neighbor is an obstacle:
                continue

            new_g = g(current) + 1

            h = Manhattan Distance(neighbor, Goal)

            f = new_g + h

            Add neighbor to OPEN

    return failure
```

---

3. Autonomous Rescue Robot – Online Search in a Partially Observable Environment
Problem Statement
An autonomous rescue robot is deployed inside a disaster-affected building.
The robot must:

- Start at position **S**
- Locate a survivor at **G**
- Move through the building
- Avoid obstacles
- Handle risky zones
- Use limited sensor information
- Dynamically update its knowledge
- Minimize total movement and risk cost
- Reach the survivor safely

This is an important example of an **AI intelligent-agent problem**.

3(a). Analysis of Constraints
Constraint 1 – Movement The
robot can move:

- Up
- Down
- Left
- Right

Diagonal movement is not allowed.
Therefore, from each cell, there can be a maximum of four neighboring cells.

Constraint 2 – Obstacles Some
cells are blocked.
The robot cannot enter those cells.
For example:
S . # . .
. # .
. . . G
Here # represents an obstacle.
The robot must identify blocked cells and select another route.

Constraint 3 – Limited Sensing
The robot can observe only nearby cells.
It does not have complete information about the entire building.
Therefore, the environment is:
\[\boxed{\text{Partially Observable}} \] The
robot must continuously gather information.

Constraint 4 – Movement Cost Each
normal movement has cost:
\[1 \]

Therefore, if the robot moves 5 normal steps:
$$ Cost=5 $$

Constraint 5 – Risky Zone Cost
Entering a risky zone has an additional cost of 2.
Therefore:
$$ Cost(\text{risky cell})=1+2=3 $$ For
example, if a path contains:

- 4 normal moves
- 2 risky-zone entries Then:

$$ Total=4+(2\times3) $$$$ =4+6 $$$$ \boxed{10} $$

Cost Function
The total path cost can be written as:
$$ \boxed{ Cost = \text{Movement Cost}+\text{Risk Cost} } $$ or:
$$ \boxed{ g(n)=\sum movement\ costs+\sum risk\ costs } $$
For a normal cell: $$ Cost=1 $$

For a risky cell: \[
Cost=3 \]

Constraint 6 – Dynamic Knowledge
The robot initially does not know the complete map.
As it moves, it receives new information from sensors.
For example:
Initially S
? ? ?
? ? ? ?
? ? G ?
? means unknown.
After sensing:
S . # ? .
. # ?
? . . G
The robot updates its internal map.

Constraint 7 – Safe Navigation
The robot should not simply choose the shortest path. It
should consider:

- Obstacles
- Risky zones
- Unknown areas
- Movement cost
- Safety
- Survivor location

Therefore, the robot needs a **rational decision-making strategy**.

3(b). Solution Approach
Because the environment is partially observable and the robot learns the map while moving, a combination of:
\[\boxed{\text{Online Search + Uniform Cost Search}} \] is
appropriate.
If a known/estimated map is available and a heuristic is desired, **A*** can also be used. But because the problem emphasizes minimizing actual cost in an initially unknown environment, **online UCS** is a strong choice.

Why Uniform Cost Search?
UCS expands the state having the lowest cumulative cost.
\[\boxed{f(n)=g(n)} \]
This is useful because different cells have different costs.
Normal cell:
\[1 \]

Risky cell:
$$3$$
UCS does not simply count the number of steps.
It considers the **actual total cost**.

Example
Suppose the robot has two paths.
Path A S → A →
B → G
All normal cells.
Cost:
$$1+1+1=3$$
Path B
S → C → D → E → G
Suppose C and D are risky.
Cost:
$$1+3+3+1=8$$
Although Path B may have a different structure, UCS chooses based on total cost:
$$3 < 8$$
Therefore, Path A is selected.

Online Search Process
The robot performs the following cycle:
Sense
↓
Update Map
↓
Generate Possible Actions
↓
Calculate Costs
↓
Select Best Action
↓
Move
↓
Sense Again
↓
Update Map
↓
Repeat

Detailed Steps
Step 1 – Start The
robot begins at:
\[S \]
It knows only the cells that its sensors can detect.

Step 2 – Sense Environment The
robot checks adjacent cells.
It identifies:

- Free cells
- Obstacles
- Risky cells
- Unknown cells

Step 3 – Update Internal Map
The robot stores the information.
For example:
S . # .
R ? ?
? G
where:

- S = Start
- G = Goal
- # = obstacle
- R = risky zone
- ? = unknown

Step 4 – Generate Possible Moves The
robot generates:

- Up
- Down
- Left
- Right

It removes:

- Blocked cells
- Invalid movements
- Unsafe actions where appropriate

Step 5 – Calculate Cost For
every valid movement:
Normal cell
\[Cost=1 \]
Risky cell

$$ Cost=3 $$

Step 6 – Use UCS

The robot places possible states into a priority queue.

The state with the lowest cumulative cost is selected.

Example:

Cell Path Cost

A 1

B 3

C 4

D 6

UCS selects:

$$ A $$ because:

$$ 1<3<4<6 $$

Step 7 – Move

The robot moves to the selected cell.

After moving, it senses the environment again.

Step 8 – Update Knowledge

If a previously unknown obstacle is discovered, the robot updates its map. For example:

Before sensing

S → ? → G

After sensing

S → # → G

The previous route is no longer possible. The robot must search for another route.

Step 9 – Continue Until Goal The process continues:

Sense
↓
Update
↓
Search
↓
Move
↓
Sense
↓
Update
↓
Search
↓
Move
↓
G

Online Search Pseudocode

```
ONLINE_RESCUE_AGENT():

    current = Start

    Initialize internal map

    WHILE current != Goal:

        percept = SenseNearbyCells()

        UpdateInternalMap(percept)

        possible_moves = GenerateMoves(current)

        RemoveBlockedMoves(possible_moves)
```

FOR each move:

IF destination is normal:
cost = 1

ELSE IF destination is risky:
cost = 3

Use Uniform Cost Search to find the least-cost known route

Select next safe action

Move to next cell

Update current position

RETURN "Survivor reached"

3(c). AI Concepts

1. Intelligent Agent

An intelligent agent is a system that:

1. Perceives its environment.
2. Processes information.
3. Makes decisions.
4. Performs actions.

The rescue robot satisfies all four.

Agent Structure

Environment
↓
Sensors
↓
Percepts
↓
Internal Knowledge
↓
Decision Making
↓
Actions
↓
Environment

Sensors

The robot uses sensors to detect:

- Walls
- Obstacles
- Risk

- Nearby rooms
- Survivors
- Paths

Actuators

The robot uses actuators to:

- Move forward
- Turn
- Stop
- Enter rooms
- Perform rescue operations

---

2. Rationality

A rational agent selects an action that maximizes expected performance based on available information.

The rescue robot should choose actions that:

- Minimize cost
- Avoid obstacles
- Reduce risk
- Reach survivors quickly
- Preserve energy
- Maintain safety

For example, suppose there are two paths:

Path A Cost: \[10 \] Risk:

Low

Path B Cost:

\[8 \] Risk:

Very high

A purely cost-based system may select B, but a safety-aware rescue robot should consider both **cost and safety**.

Therefore, the performance measure should include safety.

Performance Measure

The robot can be evaluated using:

\[\boxed{ Performance = Safety + Survivor\ Rescue + Low\ Cost + Low\ Time } \] A practical system can assign weights to these objectives.

For example:

- Survivor safety → high importance
- Robot safety → high importance
- Cost → medium importance
- Time → medium importance

---

3. Environment Type

The rescue environment has several AI environment characteristics.

Property	**Rescue Robot Environment**
Observability	Partially observable

Deterministic	Partially uncertain
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Mostly discrete grid
Number of agents	Primarily single agent
Known/Unknown	Partially unknown

Partially Observable
The robot can observe only nearby cells. It cannot see the entire building.

Sequential
One action affects future decisions.
For example:
Move → Observe → Update Map → Plan → Move

Dynamic
The disaster environment may change.
For example:

- A path may become blocked.
- A risky zone may become unsafe.
- A survivor's location may change.
- Debris may fall.

Partially Unknown
The complete building map is not initially available. The robot discovers information as it moves.

3(d). Why Online UCS Is Suitable The chosen approach is:
\[\boxed{\text{Online Search + Uniform Cost Search}} \] This is suitable for several reasons.

Reason 1 – Different Costs
Normal movements have cost 1, while risky movements have cost 3.
UCS considers actual cumulative cost.
\[g(n)=\text{total cost from Start to n} \]
Therefore, it can avoid unnecessarily expensive routes.

Reason 2 – Partial Information
The robot does not initially know the complete environment.
Online search allows the robot to:

- Move
- Sense

- Learn
- Update
- Re-plan

Reason 3 – Dynamic Environment
If the environment changes, the robot can update its internal model and search again.

Reason 4 – Safety
Blocked cells can be removed from the search space.
Risky cells can be assigned higher costs.
Therefore, the robot can prefer safer routes when their costs are lower.

Reason 5 – Optimal Cost
UCS finds the least-cost path when the relevant costs are non-negative and the required map information is available to the search. This makes it suitable for minimizing:
\[Movement + Risk \]

Complete Rescue Robot Architecture

DISASTER ENVIRONMENT
↓
SENSORS
↓
PERCEPTS
↓
INTERNAL WORLD MODEL
↓
UPDATE MAP / KNOWLEDGE
↓
GENERATE POSSIBLE ACTIONS
↓
CALCULATE ACTION COST
↓
UNIFORM COST SEARCH
↓
SELECT SAFE ACTION
↓
ACTUATORS
↓
MOVE ROBOT
↓
SENSE AGAIN
↓
UPDATE KNOWLEDGE
↓
RE-PLAN
↓

REACH SURVIVOR (G)

Comparison of Search Methods

Feature	BFS	UCS	A*	Online Search
Uses path cost	No	Yes	Yes	Yes
Uses heuristic	No	No	Yes	Can use
Handles different costs	No	Yes	Yes	Yes
Works with unknown environment	Limited	Limited	Limited	**Yes**
Dynamic replanning	Limited	Possible	Possible	**Yes**
Suitable for rescue problem	Moderate	Good	Good	**Very suitable**

Final Answers at a Glance
Question 1
Final Schedule

\[\boxed{ D1\rightarrow Morning } \]\[\boxed{ D2\rightarrow Afternoon } \]\[\boxed{ D3\rightarrow Night } \]
All constraints are satisfied.

Question 2

Using the **given example 5×5 grid**, the Manhattan-distance search gives:
\[\boxed{ S\rightarrow(1,2)\rightarrow(2,2)\rightarrow(3,2) \rightarrow(3,3)\rightarrow(4,3)\rightarrow(5,3) \rightarrow(5,4)\rightarrow G } \]
Total movements: \[\boxed{8} \] Total cost:
\[\boxed{8} \]
Important: If your college question has a **specific 5×5 grid/obstacle figure**, send that figure. The node-expansion table and final path must be recalculated according to those exact obstacles.

Question 3

The rescue robot should use an **online search approach**, with **Uniform Cost Search** for costsensitive route selection.
Normal movement:
\[\boxed{1} \]
Risky-zone movement:
\[\boxed{1+2=3} \] The
robot repeatedly:
\[\boxed{ Sense\rightarrow Update\rightarrow Search\rightarrow Move\rightarrow Repeat } \]